

10-00 — TypeScript for Node.js Developers

Position

Bridge lecture — between Section 9 (Views & Static Assets) and Section 10 (TypeORM Introduction & Setup)

Audience

Developers who have completed Sections 1–9 in JavaScript. You have written Node.js servers, Express routes, and middleware — all in plain JS. This lecture moves that same knowledge into TypeScript, specifically the subset TypeORM requires.

Est. Duration

60 – 90 minutes

Type

Theory

Practical

Bridge

Topics covered

Why TypeScript, type erasure, compilation pipeline, project setup, `tsconfig.json` options explained, type annotations, interfaces, classes with access modifiers, constructor shorthand, union types, ES module imports, decorators, `reflect-metadata`, TypeORM transition

Learning Objectives — This Lecture

- Articulate *why* TypeScript matters in a Node.js project beyond the surface-level answer of "type safety".
- Explain what "type erasure" means and why it has practical consequences for runtime behaviour.
- Set up a TypeScript Node.js project from scratch and justify the purpose of every installed package and every relevant `tsconfig.json` option.

- Apply type annotations to variables and functions; distinguish annotation from inference and know when each is appropriate.
- Write interfaces and explain why they are different from classes at the language level.
- Write TypeScript classes with property declarations and access modifiers, using constructor shorthand where appropriate.
- Use ES-module `import / export` syntax in a CommonJS Node.js project and explain how `esModuleInterop` connects the two.
- Write a working class decorator, explain when it executes, and connect that mechanism to what TypeORM's `@Entity()` and `@Column()` decorators do.

1. Why Does Node.js Need TypeScript?

JavaScript is **dynamically typed**. That means variables have no declared type — they can hold any value, and the language only checks types when the code actually runs. For small scripts, this is fine. For a production Node.js server handling hundreds of routes, it creates a specific class of problem: bugs that are completely invisible until they surface in a running application.

1.1 The Problem TypeScript Solves

Consider a function that adds two numbers — something you have written many times in plain JavaScript:

JAVASCRIPT

```
function add(a, b) {  
  return a + b;  
}  
  
console.log(add(5, "10"));
```

WHAT YOU EXPECTED

```
// Expected: 15  
// Actual:   "510"  
  
// JavaScript silently coerces the number 5  
// into a string and concatenates.  
// No error. No warning. Wrong answer.
```

Output:

```
510 ← JavaScript output: wrong, but no crash, no warning
```

This particular example is easy to spot. The dangerous version is the same bug buried inside a 400-line route handler, triggered only by a specific combination of query parameters that appear in production three weeks after launch.

TypeScript prevents this entire category of mistake by checking types *before the code runs*:

src/math.ts

```
function add(a: number, b: number): number {  
  return a + b;  
}  
  
add(5, "10");  
// TypeScript Error TS2345:  
// Argument of type 'string' is not assignable to parameter of type 'number'.
```

Output:

Compile error – the programme will not run at all until this is fixed.
The bug is caught at the point it is written, not when a user triggers it in production.

Why this matters for Node.js specifically: A browser bug affects one user who then refreshes the page. A Node.js server bug affects every concurrent request — and since the error only surfaces at runtime, it has often already served corrupted data or crashed the process by the time you know about it. TypeScript moves the detection point from "runtime, in production" to "compile time, on your machine."

1.2 The Full Picture: What TypeScript Gives You

Benefit	What it means in your Node.js codebase
Errors before runtime	Type mismatches, typos on property names, wrong argument order — all caught before <code>node</code> executes a single line.
Editor autocomplete on your own code	

Benefit	What it means in your Node.js codebase
	Your editor knows what properties a <code>User</code> object has. In plain JS it guesses; in TypeScript it knows. Navigating a codebase becomes dramatically faster.
Self-documenting signatures	<pre>function createUser(dto: CreateUserDto): Promise<User></pre> tells the next developer everything about what this function takes and returns — no need to read the body.
Safer refactoring	Rename a property on an interface — TypeScript immediately highlights every usage of the old name across the entire codebase.
TypeORM entity typing	When you fetch a <code>Product</code> from the database, TypeScript knows exactly what properties it has and their types. Accessing a non-existent property is a compile error, not a silent <code>undefined</code> .

Key Insight — TypeScript is the foundation, not the goal: TypeScript itself is not what you are building. It is the foundation that TypeORM, NestJS, Prisma, and most modern backend frameworks are built on. Once you are comfortable with classes, decorators, and modules in TypeScript, those frameworks stop looking like magic — their patterns become readable and predictable.

2. The Most Important Concept: Type Erasure

Before writing a single line of TypeScript, you need to understand one thing: **TypeScript does not exist at runtime.**

TypeScript is a *compile-time layer*. It reads your annotated `.ts` source files, checks them for correctness, and then strips every type annotation out to produce standard JavaScript. What runs on your Node.js server is plain JavaScript — the V8 engine has no concept of TypeScript types whatsoever.

Analogy: TypeScript is like a proof-reader who checks a document before it goes to print. The proof-reader's notes and corrections are not printed in the final document — only the corrected text goes to press. TypeScript's types are the proof-reader's notes: essential during the writing process, completely absent in the final output.

2.1 Seeing Type Erasure Happen

The best way to internalise this is to compile a file and compare the input to the output. Start with a file that uses several TypeScript features:

TYPESCRIPT SOURCE — SRC/INDEX.TS

```
let port: number = 3000;
let host: string = "localhost";

interface ServerConfig {
  port: number;
  host: string;
}

function startServer(config: ServerConfig):
void {
  console.log(
    `Starting on ${config.host}:${
config.port}`
  );
}

startServer({ port, host });
```

COMPILED OUTPUT — DIST/INDEX.JS

```
"use strict";
let port = 3000;
let host = "localhost";

// interface ServerConfig → gone entirely

function startServer(config) { // ":
ServerConfig" → gone
  console.log(
    `Starting on ${config.host}:${
config.port}`
  );
} // ": void" →
gone

startServer({ port, host });
```

Every `: number`, `: string`, `: ServerConfig`, `: void` — gone. The entire `interface ServerConfig { ... }` block — gone. The compiled JavaScript is structurally identical to what you would have written in plain JS.

Why this has a practical consequence: Because types disappear at runtime, you cannot use them for runtime logic. There is no way to write `if (x instanceof ServerConfig)` — `ServerConfig` is an interface and does not exist at runtime. For runtime shape-validation (e.g. checking that a request body has the right fields) you still need JavaScript tools: `typeof`, `instanceof`, or a library like `joi` or `zod`. TypeORM solves this problem differently — using *decorators* and *metadata*, which do persist at runtime. That is what Section 7 of this lecture covers.

3. Project Setup — Every Package Explained

In Sections 1–9 you ran Node.js applications by executing `.js` files directly with `node filename.js`. TypeScript introduces a compilation step between writing and running. The setup below configures two workflows: a fast development loop using `ts-node`, and a production build using the TypeScript compiler.

3.1 Initialise the Project

```
mkdir ts-node-demo && cd ts-node-demo
```

```
npm init -y
```

Output:

Wrote to /path/to/ts-node-demo/package.json

3.2 Install the Toolchain

```
npm install --save-dev typescript ts-node nodemon @types/node
```

```
npm install reflect-metadata
```

Output:

added 5 packages

Each package has a specific reason for being here. Understanding what each one does — and why it is needed — prevents confusion when something does not work.

Why `typescript` ?

Node.js cannot run `.ts` files. It only understands JavaScript. The `typescript` package provides the `tsc` compiler — the programme that translates your TypeScript source into the JavaScript that Node.js can execute.



This is the production workflow. It is correct, but slow for development — you would need to run `tsc` after every change before you could test anything.

Why `ts-node` ?

`ts-node` collapses the compile-then-run cycle into a single step. It hooks into Node.js's module loader and compiles each `.ts` file on-the-fly as it is first `require()` d, without writing any `.js` files to disk.



Important: `ts-node` is a development tool only. It adds overhead on every startup (compiling on-the-fly takes time and memory) and should never be used in production. Production servers always run pre-compiled JavaScript from `dist/`.

Why `nodemon` ?

Nodemon watches your source files for changes and automatically restarts the process. You have already used this in the module with plain JavaScript. It does the same job here — the only difference is that it now calls `ts-node` instead of `node` when restarting.

Without nodemon, the development loop is:

Save file → Ctrl+C → run command again → wait → test → repeat. With nodemon: *Save file → test.*

Why `@types/node` ?

TypeScript needs to know the type of everything you use. Node.js built-in modules like `fs` , `path` , `http` , `process` , and `Buffer` are not part of the browser DOM types that TypeScript ships with. Without this package, writing `import fs from 'fs'` produces:

Output:

```
TS2307: Cannot find module 'fs' or its corresponding type declarations.
```

`@types/node` is a package that contains *only* TypeScript type definition files (`.d.ts` files) — no JavaScript. It describes the shape of every Node.js API to TypeScript. Because it is pure type information, it contributes nothing to the running programme and belongs in `devDependencies` .

The `@types/` pattern: Most JavaScript libraries predate TypeScript. Their npm packages contain no type information. The community-maintained `@types/` namespace provides type definitions separately. When you later install Express for a TypeScript project, you will also install `@types/express` alongside it for the same reason.

Why `reflect-metadata` ?

TypeScript types are erased at runtime (Section 2). TypeORM needs to know the TypeScript type of each entity property at runtime — for example, "this `name` property has type `string` , so I should create a `VARCHAR` column." `reflect-metadata` provides an API (`Reflect.defineMetadata` / `Reflect.getMetadata`) that lets libraries store and retrieve arbitrary metadata attached to classes and properties. TypeORM uses this to preserve type information that would otherwise be erased. The full mechanism is explained in Section 7.

Unlike the others, `reflect-metadata` is a runtime dependency — not a dev dependency — because the running application needs it. TypeORM will throw an error on startup if it is missing.

3.3 Create and Understand `tsconfig.json`

```
npx tsc --init
```


Output:

Created a new tsconfig.json with ~100 options (most commented out)

Replace the generated file's contents with this focused configuration. Every option here is deliberate:

tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "CommonJS",
    "rootDir": "./src",
    "outDir": "./dist",
    "strict": true,
    "esModuleInterop": true,
    "experimentalDecorators": true,
    "emitDecoratorMetadata": true
  }
}
```

Why each option is set to this value:

Option	Value	Why
target	"ES2020"	The JavaScript version the compiler outputs. <code>ES2020</code> supports <code>async/await</code> , optional chaining, nullish coalescing natively — appropriate for any Node.js 14+. Choosing <code>ES5</code> would force the compiler to polyfill modern syntax into verbose callback chains unnecessarily.
module	"CommonJS"	Node.js uses CommonJS (<code>require</code> / <code>module.exports</code>) by default. Even though you

Option	Value	Why
		write <code>import / export</code> in TypeScript, the compiler converts them to <code>require()</code> calls in the output. This is what Sections 1–9 used throughout, so nothing changes under the hood.
<code>rootDir</code>	<code>"./src"</code>	All TypeScript source files must be inside <code>src/</code> . The compiler refuses to compile files outside this folder.
<code>outDir</code>	<code>"./dist"</code>	Where the compiled <code>.js</code> files go. Keeping source (<code>src/</code>) and compiled output (<code>dist/</code>) separate is standard practice. Add <code>dist/</code> to <code>.gitignore</code> — it is generated output, not source code.
<code>strict</code>	<code>true</code>	Enables a bundle of strict checks. The two most significant are: <code>strictNullChecks</code> (prevents treating <code>null</code> / <code>undefined</code> as valid values of any type unless explicitly declared) and <code>noImplicitAny</code> (forces you to be explicit when TypeScript cannot infer a type). Always enable this — turning it off to suppress errors hides real bugs.

Option	Value	Why
<code>esModuleInterop</code>	<code>true</code>	Allows <code>import express from 'express'</code> for CommonJS packages that do not have a proper default export property. Without it, importing Express or similar packages requires the awkward <code>import * as express from 'express'</code> syntax everywhere.
<code>experimentalDecorators</code>	<code>true</code>	Enables the <code>@Decorator</code> syntax. TypeORM's entire mapping system — <code>@Entity()</code> , <code>@Column()</code> , <code>@OneToMany()</code> — uses decorators. Without this option, the TypeScript compiler rejects decorator syntax.
<code>emitDecoratorMetadata</code>	<code>true</code>	Instructs the compiler to emit extra <code>Reflect.metadata()</code> calls into the compiled JavaScript, recording the TypeScript type of each decorated property. TypeORM reads this at runtime to determine column types. Without it, TypeORM cannot infer that a <code>name: string</code> property should be a <code>VARCHAR</code> column — you would need to specify the database type manually on every single <code>@Column()</code> .

Why `rootDir` and `outDir` matter: Without these, the compiled JavaScript files land next to your TypeScript source files — you end up with both `src/index.ts` and `src/index.js` in the same directory. This quickly becomes confusing. The `rootDir` / `outDir` separation keeps the project clean: `src/` is always source, `dist/` is always compiled output. Production servers run `node dist/index.js` — they never touch `src/`.

3.4 Add npm Scripts

package.json (scripts section)

```
{
  "scripts": {
    "dev": "nodemon --watch src --ext ts --exec ts-node src/index.ts",
    "build": "tsc",
    "start": "node dist/index.js"
  }
}
```

Output:

Three distinct workflows:

```
npm run dev    → watches src/*.ts, restarts via ts-node on every save (development)
npm run build  → runs tsc, writes compiled .js to dist/ (production build)
npm start     → runs the pre-compiled dist/index.js (production server)
```

The `--ext ts` flag tells nodemon to watch only `.ts` files. Without it, nodemon defaults to watching `.js` files — and since `ts-node` does not write any `.js` files to disk in the dev workflow, saves would never trigger a restart.

3.5 First Run

STEP 1 Create `src/index.ts` with a single line:

src/index.ts

```
console.log("Hello from TypeScript + Node.js!");
```

```
npm run dev
```

Output:

```
[nodemon] starting `ts-node src/index.ts`  
Hello from TypeScript + Node.js!  
[nodemon] clean exit - waiting for changes before restart
```

The project structure at this point:

```
ts-node-demo/  
├─ src/  
│   └─ index.ts      ← TypeScript source (you edit this)  
├─ node_modules/  
├─ package.json  
└─ tsconfig.json  
  
(dist/ does not exist yet – it appears only after npm run build)
```

4. Type Annotations on Variables and Functions

Everything you have written in Node.js throughout Sections 1–9 is valid TypeScript — JavaScript is a subset of TypeScript. What TypeScript adds is the ability to *annotate* constructs with types. The annotation syntax is a colon after the variable name or parameter, followed by the type name.

4.1 Variable Annotations and Inference

STAGE 1 Annotating variables explicitly.

```
src/index.ts
```

```
let port: number    = 3000;
let host: string    = "localhost";
let isDev: boolean  = true;

console.log(`${host}:${port} - dev mode: ${isDev}`);
```

Output:

```
localhost:3000 - dev mode: true
```

STAGE 2 TypeScript can infer the type when the initial value makes it obvious. The annotation is then redundant — you do not need to write it.

src/index.ts

```
// Annotated – correct but redundant
let port: number = 3000;

// Inferred – TypeScript reads the literal 3000 and infers "number" automatically
let port2 = 3000;

// Both are identical. Hover over "port2" in VS Code – it shows: let port2: number

// Where inference CANNOT help – an empty array has no inferred item type
let results: string[] = []; // must annotate – TypeScript cannot know what goes in here
```

Output:

No error – all declarations are valid. TypeScript infers from literal values automatically.

When to annotate vs when to rely on inference: A practical rule: always annotate function parameter and return types (inference cannot know what callers will pass), and annotate variables when the initial value does not make the type obvious (e.g. an empty array, a `null` initial value, or a variable declared before its value is assigned). Let inference handle everything else — over-annotating adds verbosity without safety.

4.2 Function Annotations — Where TypeScript Provides the Most Value

STAGE 3 Annotating a function turns its signature into a contract. Any caller that violates the contract gets a compile error.

src/index.ts

```
function buildUrl(host: string, port: number, path: string): string {  
  return `http://${host}:${port}${path}`;  
}  
  
// Correct usage  
console.log(buildUrl("localhost", 3000, "/api/users"));
```

Output:

http://localhost:3000/api/users

src/index.ts (demonstrating the error)

```
// Accidentally swapped the port and path arguments:  
buildUrl("localhost", "/api/users", 3000);  
// Error TS2345: Argument of type 'string' is not assignable to parameter of type 'number'.  
// Error TS2345: Argument of type 'number' is not assignable to parameter of type 'string'.  
  
// In plain JavaScript this silently produces:  
// "http://localhost:/api/users3000" ← wrong, but no crash
```

Output:

Compile error — caught before any code runs.

4.3 Union Types

A union type says a value can be one of several types. This is common in Express route handlers — `req.params.id` is always a `string` (URL parameters are text), even when the underlying database column is an integer. A function that accepts either is explicit about that flexibility:

src/index.ts

```
// Accepts either a number (from the database) or a string (from req.params)
function findById(id: number | string): string {
  return `SELECT * FROM products WHERE id = ${id}`;
}

console.log(findById(42));
console.log(findById("42"));
```

Output:

```
SELECT * FROM products WHERE id = 42
SELECT * FROM products WHERE id = 42
```

5. Interfaces

An interface describes the *shape* of an object — which properties it has and what types they are. Interfaces are compile-time only: they are completely erased and have no runtime representation.

Why interfaces and not just plain objects? Without an interface, TypeScript types function parameters as `any` when the shape is not specified. `any` disables all type checking — it is the escape hatch that tells TypeScript "stop checking this value." The moment you use `any`, all protection disappears: property typos, accessing non-existent fields, and wrong types all compile silently.

5.1 The Problem `any` Creates

WITHOUT INTERFACE (BODY TYPED AS ANY)

```
function register(dto: any): string {
  // "emal" is a typo – should be "email"
  // TypeScript says nothing. It trusts any.
  return `${dto.name} (${dto.emal})`;
}
```

WITH INTERFACE

```
interface CreateUserDto {
  name: string;
  email: string;
}
function register(dto: CreateUserDto): string
{
  return `${dto.name} (${dto.email})`;
}
```



```
// Output: "Alice (undefined)"  
// Silent wrong data in production
```

```
// Error TS2551: Property 'emal' does not  
// exist on type 'CreateUserDto'.  
// Did you mean 'email'?  
}
```

Output:

Compile error on the right – caught before running.

TypeScript even suggests the correct property name in the error message.

5.2 Defining and Using an Interface

src/index.ts

```
interface CreateUserDto {  
  name: string;  
  email: string;  
  age?: number;    // the ? makes the property optional  
}  
  
function registerUser(dto: CreateUserDto): string {  
  const ageInfo = dto.age !== undefined ? `, age ${dto.age}` : "";  
  return `Registered: ${dto.name} (${dto.email}${ageInfo})`;  
}  
  
// Required fields only  
console.log(registerUser({ name: "Alice", email: "alice@example.com" }));  
  
// Optional field included  
console.log(registerUser({ name: "Bob", email: "bob@example.com", age: 28 }));
```

Output:

```
Registered: Alice (alice@example.com)  
Registered: Bob (bob@example.com, age 28)
```

The abbreviation *DTO* stands for Data Transfer Object — a plain object whose only job is to carry data between layers. You will use this pattern extensively: the request body arrives as raw JSON, you describe its expected shape with a DTO interface, and TypeScript ensures your route handler only accesses properties that are actually declared.

5.3 Interfaces Are Not Classes

This is a distinction that trips up many developers coming from languages where "interface" implies something more substantial. In TypeScript, an interface is purely a compiler instruction — it compiles to nothing. A class, by contrast, compiles to a JavaScript constructor function that exists at runtime, can be instantiated with `new`, and supports `instanceof` checks.

	Interface	Class
Exists at runtime?	No — erased entirely	Yes — becomes a JS constructor
<code>instanceof</code> check?	No	Yes
Can have method bodies?	No — declarations only	Yes
Primary use in this module	DTOs, request/response shapes	TypeORM entities, services

6. Classes and Access Modifiers

You have used JavaScript classes in Sections 1–9. TypeScript adds two things to them: property type declarations (you must declare properties before using them) and access modifiers (`public`, `private`, `protected`). TypeORM entities are TypeScript classes — this section is a direct prerequisite for everything in Sections 10–14.

6.1 Property Declarations — Why You Cannot Skip Them

In plain JavaScript you can assign properties to `this` inside a constructor without declaring them first at the class level. TypeScript's `strict` mode requires you to declare every property explicitly:

JAVASCRIPT (WORKS)

```
class Product {
  constructor(name, price) {
    this.name = name;    // fine - JS adds it
```

TYPESCRIPT WITH STRICT (REQUIRES DECLARATION)

```
class Product {
  name: string;    // declared here first
  price: number;   // declared here first
```

```
dynamically
  this.price = price; // fine
}
```

```
constructor(name: string, price: number) {
  this.name = name;
  this.price = price;
}
```

Why TypeScript requires this: Without an explicit declaration, TypeScript cannot know what type a property is — and without knowing the type, it cannot check any code that accesses it. The declaration is also exactly what TypeORM reads: when you write `name: string` on an entity class and decorate it with `@Column()`, TypeORM uses both the decorator and the type declaration to decide what kind of database column to create.

6.2 Access Modifiers

STAGE 1 TypeScript adds three access modifiers to classes.

src/models/UserAccount.ts

```
export class UserAccount {
  public email: string;           // accessible from anywhere – the default
  private passwordHash: string; // accessible only within this class
  protected role: string;        // accessible within this class and subclasses

  constructor(email: string, passwordHash: string, role: string) {
    this.email = email;
    this.passwordHash = passwordHash;
    this.role = role;
  }

  public verifyPassword(plain: string): boolean {
    return plain === this.passwordHash; // can access private from inside the class
  }
}
```

src/index.ts

```
import { UserAccount } from "../models/UserAccount";

const account = new UserAccount("alice@example.com", "hashed123", "admin");

console.log(account.email); // OK – public
console.log(account.verifyPassword("hashed123")); // OK – public method

// console.log(account.passwordHash);
// Error TS2341: Property 'passwordHash' is private and only accessible
// within class 'UserAccount'.
```

Output:

```
alice@example.com
true
```

Important — access modifiers are compile-time only: TypeScript's `private` does not produce a truly private property in the compiled JavaScript. If you inspect the compiled output, the property is fully accessible. This is distinct from JavaScript's `#propertyName` syntax (ES2022 private fields), which creates a runtime private field. TypeScript's `private` exists to catch accidental access in your TypeScript codebase — it is not a security mechanism.

6.3 Constructor Shorthand

Writing the property declaration, the constructor parameter, and the assignment separately is repetitive. TypeScript provides a shorthand: add an access modifier to a constructor parameter and TypeScript generates all three automatically.

VERBOSE FORM (THREE STEPS)

```
class Product {
  name: string;
  price: number;

  constructor(name: string, price: number) {
    this.name = name;
    this.price = price;
  }
}
```

SHORTHAND (ONE STEP, IDENTICAL RESULT)

```
class Product {
  constructor(
    public name: string,
    public price: number,
  ) {}
  // TypeScript generates the declaration
  // and assignment automatically
}
```

src/index.ts

```
class Product {
  constructor(
    public name: string,
    public price: number,
    private sku: string,
  ) {}

  public describe(): string {
    return `${this.name} - £${this.price.toFixed(2)} (SKU: ${this.sku})`;
  }
}

const p = new Product("Wireless Headphones", 79.99, "WH-001");
console.log(p.describe());
console.log(p.name); // OK - public

// console.log(p.sku); // Error TS2341: 'sku' is private
```

Output:

```
Wireless Headphones - £79.99 (SKU: WH-001)
Wireless Headphones
```

7. Modules: Import and Export in TypeScript

In Sections 1–9 you used CommonJS modules: `const express = require('express')` and `module.exports = router`. TypeScript uses ES module syntax — `import` and `export` — in your source files. Because `"module": "CommonJS"` is set in `tsconfig.json`, the compiler automatically converts these to `require()` calls in the compiled JavaScript. You write modern syntax; the output is still compatible CommonJS.

7.1 Named Exports

src/utils/http.ts

```
export function buildUrl(host: string, port: number, path: string): string {  
  return `http://${host}:${port}${path}`;  
}  
  
export const DEFAULT_PORT: number = 3000;
```

src/index.ts

```
import { buildUrl, DEFAULT_PORT } from "./utils/http";  
  
console.log(buildUrl("localhost", DEFAULT_PORT, "/api/products"));
```

Output:

http://localhost:3000/api/products

TypeScript understands what you import. If you call `buildUrl(3000, "localhost", "/api")` with the arguments in the wrong order, the compiler catches the type mismatch immediately — because it knows the exact signature of the imported function.

7.2 Exporting Interfaces and Classes

Interfaces and classes are exported exactly like functions. This is the standard pattern for sharing data shapes and entity definitions across a TypeScript project:

src/types/dtos.ts

```
export interface CreateProductDto {  
  name: string;  
  price: number;  
  description?: string;  
}  
  
export interface ProductResponse {  
  id: number;  
  name: string;
```

```
price: number;
createdAt: Date;
}
```

src/index.ts

```
import { CreateProductDto, ProductResponse } from "../types/dtos";

function createProduct(dto: CreateProductDto): ProductResponse {
  return {
    id: 1,
    name: dto.name,
    price: dto.price,
    createdAt: new Date(),
  };
}

const product = createProduct({ name: "Wireless Headphones", price: 79.99 });
console.log(`Created: ${product.name} at £${product.price}`);
```

Output:

Created: Wireless Headphones at £79.99

7.3 What the Compiler Does to Imports

It is worth seeing once what TypeScript actually emits for import statements, so the compilation step is not a mystery:

TYPESCRIPT SOURCE

```
import { buildUrl } from "../utils/http";
import { CreateProductDto } from "../types/dtos";

// CreateProductDto is an interface – erased entirely.
// Only buildUrl, which has a runtime value, survives.
console.log(buildUrl("localhost", 3000, "/"));
```

COMPILED JAVASCRIPT (DIST/)

```
"use strict";
Object.defineProperty(exports, "__esModule", { value: true });
const http_1 = require("../utils/http");
// No require for dtos.ts – interfaces have no runtime value

console.log((0, http_1.buildUrl)("localhost", 3000, "/"));
```

Output:

The interface import vanishes entirely from the compiled output.
The function import becomes a standard CommonJS require().

8. Decorators and `reflect-metadata` — The Bridge to TypeORM

This section explains the two TypeScript features that make TypeORM work. Decorators are the `@Something` syntax you will see on every TypeORM entity. `reflect-metadata` is the package that makes type information available at runtime. Together they solve a problem that is fundamental to what TypeORM does.

8.1 The Problem: Type Information Disappears

Consider what TypeORM needs to do when it encounters this class:

`src/entity/Product.ts` (hypothetical, no decorators)

```
class Product {  
  id: number;  
  name: string;  
  price: number;  
}
```

TypeORM needs to know: "create a table called `products`. Give it an integer primary key called `id`, a varchar column called `name`, and a decimal column called `price`." But in the compiled JavaScript output, the class body is empty — there are no property declarations, no types, nothing TypeORM can read. The type information was erased.

Decorators and `reflect-metadata` together solve this by attaching persistent metadata to the class at the point it is defined.

8.2 What a Decorator Is

A decorator is a function that TypeScript calls on the class (or property, or method) it is attached to, at the moment the class is *defined* — not when instances are created with `new`. The `@` prefix is TypeScript syntax for applying a decorator.

Analogy: Think of a decorator as a label you stick on a box when you pack it — before it is ever opened. When someone opens the box later, they read the label to know what is inside without unpacking everything. TypeORM sticks metadata labels on classes as they are loaded, then reads those labels when setting up the database connection.

8.3 A Working Class Decorator

Install `reflect-metadata` first if you have not already: `npm install reflect-metadata`

STAGE 1 Create a simple decorator that stores a table name on a class.

src/decorators/entity.ts

```
// A decorator factory – a function that RETURNS the actual decorator.
// This pattern lets callers pass arguments: @Entity("products")
export function Entity(tableName: string) {
  return function (constructor: Function): void {
    // This inner function IS the decorator.
    // TypeScript calls it with the class constructor as its argument.
    // It runs when the class definition is evaluated – before any instance is created.

    console.log(`@Entity applied to class: ${constructor.name}`);
    console.log(` → Table name: ${tableName}`);

    // Store the table name on the class using the reflect-metadata API.
    // This metadata persists at runtime – unlike TypeScript types, it is NOT erased.
    Reflect.defineMetadata("tableName", tableName, constructor);
  };
}
```

src/index.ts

```
import "reflect-metadata";           // must be the very first import
import { Entity } from "../decorators/entity";

@Entity("products")                  // applies the decorator to the class below
class Product {
  id: number = 0;
  name: string = "";
  price: number = 0;
}

// The decorator ran when the class was defined above – before this line executes.
// Now read back the metadata that the decorator stored:
const tableName = Reflect.getMetadata("tableName", Product);
console.log(`Stored table name: ${tableName}`);

// Creating an instance does NOT re-run the decorator
const p = new Product();
p.name = "Wireless Headphones";
console.log(`Product name: ${p.name}`);
```

Output:

```
@Entity applied to class: Product
  → Table name: products
Stored table name: products
Product name: Wireless Headphones
```

Notice that the decorator output appears before "Stored table name" — because the decorator ran when the class was defined (the `@Entity("products")` line), not when `Reflect.getMetadata` was called. This is the key behaviour TypeORM relies on: all entity classes are loaded when the application starts, their decorators fire, and TypeORM collects all the metadata before any database query is made.

8.4 A Working Property Decorator — Reading TypeScript Types at Runtime

STAGE 2 The second critical piece: reading the TypeScript type of a property at runtime, using the metadata the compiler emits when `emitDecoratorMetadata: true` is set.

```
src/decorators/column.ts
```

```
export function Column(target: Object, propertyKey: string): void {
  // target      → the class prototype
  // propertyKey → the name of the decorated property ("name", "price", etc.)

  // "design:type" is a special metadata key emitted by the TypeScript compiler
  // when emitDecoratorMetadata is true. It stores the JavaScript constructor
  // function for the TypeScript type: String, Number, Boolean, etc.
  const typeConstructor = Reflect.getMetadata("design:type", target, propertyKey);

  // Retrieve any previously registered columns and append this one
  const columns: string[] = Reflect.getMetadata("columns", target.constructor) || [];
  columns.push(`${propertyKey}: ${typeConstructor.name}`);
  Reflect.defineMetadata("columns", columns, target.constructor);
}
```

src/index.ts

```
import "reflect-metadata";
import { Entity } from "../decorators/entity";
import { Column } from "../decorators/column";

@Entity("products")
class Product {
  @Column
  id: number = 0;      // emitDecoratorMetadata stores: "design:type" → Number

  @Column
  name: string = "";   // emitDecoratorMetadata stores: "design:type" → String

  @Column
  price: number = 0;

  @Column
  isActive: boolean = true; // emitDecoratorMetadata stores: "design:type" → Boolean
}

const tableName = Reflect.getMetadata("tableName", Product);
const columns    = Reflect.getMetadata("columns",    Product);

console.log(`\nSimulated schema for table: ${tableName}`);
columns.forEach((col: string) => console.log(`  column: ${col}`));
```

Output:

```
@Entity applied to class: Product
  → Table name: products
```

```
Simulated schema for table: products
column: id: Number
column: name: String
column: price: Number
column: isActive: Boolean
```

This is exactly what TypeORM's actual `@Entity()` and `@Column()` do — the real implementation is more elaborate, but the mechanism is identical. When `AppDataSource.initialize()` runs, TypeORM walks all registered entity classes, reads all the metadata stored by their decorators, and generates the SQL DDL.

The complete chain — why all four pieces must be in place: (1) You write `name: string` with `@Column`. (2) The TypeScript compiler, because `emitDecoratorMetadata: true`, emits a hidden `Reflect.metadata("design:type", String)` call alongside the compiled property decorator call. (3) `reflect-metadata` provides the `Reflect` API to store and read this data. (4) TypeORM's `@Column()` reads the type via `Reflect.getMetadata("design:type", ...)` and maps `String` → `VARCHAR`. Remove any one of these and the chain breaks.

8.5 Why `import "reflect-metadata"` Must Be First

`reflect-metadata` patches the global `Reflect` object with its metadata API. If anything tries to call `Reflect.defineMetadata` before this import runs, it will throw a runtime error because the method does not exist yet. Always place it as the very first import in your application entry point — before any entity, decorator, or TypeORM import.

src/index.ts – correct order

```
import "reflect-metadata";    // ← FIRST – patches global Reflect
import { AppDataSource } from "../data-source";    // ← safe to import entities now
import { Product } from "../entity/Product";    // ← entity decorators can fire
```

Output:

Correct – reflect-metadata is ready before any entity decorator tries to call Reflect.defineMetadata()

9. Everything Maps Directly to TypeORM

Every concept covered in this lecture has a direct counterpart in TypeORM. The syntax in Sections 10–14 should now look recognisable rather than arbitrary.

What you learned → What TypeORM uses it for

- **Classes with property declarations** → TypeORM entities are TypeScript classes. `@Entity()` is a class decorator. Every entity property with `@Column()` must be declared with its TypeScript type.
- **Access modifiers** → Entity properties are `public` by default so TypeORM can read and write them. Service and repository methods may use `private` helpers internally.
- **Interfaces** → DTOs for route handlers. The incoming `req.body` is typed against a DTO interface; the saved entity is a class instance. The two are kept separate deliberately.
- **Import / export** → Each entity lives in its own file and is exported with `export class`. The `DataSource` imports and registers all entity classes. Repositories and services import the entities they work with.
- **Decorators** → `@Entity()`, `@PrimaryGeneratedColumn()`, `@Column()`, `@OneToMany()` — TypeORM's entire mapping system is decorator-based.
- **reflect-metadata** → Must be the first import in `data-source.ts` or the entry point. Without it, TypeORM throws "reflect-metadata shim is required" on startup.

src/entity/Product.ts – a real TypeORM entity using everything from this lecture

```
// reflect-metadata imported once at the application entry point

import { Entity, PrimaryGeneratedColumn, Column } from "typeorm";
//      ↑ class decorator   ↑ property decorator   ↑ property decorator
```

```

@Entity("products")           // class decorator – stores table name as metadata
export class Product {       // class – with property declarations (Section 6)

  @PrimaryGeneratedColumn()   // property decorator – stores "this is the PK" as metadata
  id: number;                 // TypeScript type – compiler emits "design:type": Number

  @Column({ length: 120 })    // property decorator with options
  name: string;              // "design:type": String → TypeORM maps to VARCHAR

  @Column("decimal", { precision: 10, scale: 2 })
  price: number;             // "design:type": Number → TypeORM maps to DECIMAL

  @Column({ default: true })
  isActive: boolean;         // "design:type": Boolean → TypeORM maps to BOOLEAN / TINYINT
}
// export class – ES module syntax (Section 7)
// DataSource imports this class and reads all its decorator metadata

```

Output:

When `AppDataSource.initialize()` runs, TypeORM reads the metadata and generates:

```

CREATE TABLE products (
  id      integer      PRIMARY KEY AUTOINCREMENT NOT NULL,
  name    varchar(120) NOT NULL,
  price   decimal(10,2) NOT NULL,
  isActive boolean     NOT NULL DEFAULT true
);

```

Summary

Tool / Concept	Why it exists
TypeScript	Catches type errors before running. Better autocomplete, safer refactoring, self-documenting signatures.
Type erasure	

Tool / Concept	Why it exists
	All types are stripped from compiled output. Runtime behaviour is plain JavaScript. Types cannot be used for runtime checks.
<code>tsc</code>	The TypeScript compiler. Reads <code>.ts</code> files, checks types, writes <code>.js</code> files to <code>dist/</code> . Used for production builds.
<code>ts-node</code>	Compiles and runs TypeScript in-memory without writing to disk. Development only — adds startup overhead.
<code>nodemon</code>	Watches <code>src/</code> for changes and auto-restarts the process. Same role as in the JS sections of this module.
<code>@types/node</code>	Type definitions for Node.js built-ins (<code>fs</code> , <code>path</code> , <code>http</code> , <code>process</code>). Dev dependency — no runtime presence.
<code>tsconfig.json</code>	Compiler configuration. <code>rootDir</code> / <code>outDir</code> separate source from output. <code>strict</code> enables meaningful checks. <code>experimentalDecorators</code> + <code>emitDecoratorMetadata</code> enable TypeORM.
Type annotations	Colons after names: <code>let x: number</code> , <code>function f(a: string): void</code> . Catch mismatches at compile time.
Type inference	TypeScript infers types from literal values. Annotate function signatures always; let inference handle obvious variable initialisation.
Interfaces	Compile-time object shapes. Erased entirely — no runtime existence. Use for DTOs and request/response shapes.

Tool / Concept	Why it exists
Access modifiers	<code>public</code> / <code>private</code> / <code>protected</code> . Compile-time enforcement only — not a runtime security mechanism.
Constructor shorthand	<code>constructor(public name: string) {}</code> — declares and assigns the property in one step.
ES module imports	Write <code>import</code> / <code>export</code> ; compiler converts to <code>require()</code> when <code>"module": "CommonJS"</code> is set.
Decorators	Functions applied with <code>@</code> that run at class-definition time and attach metadata. TypeORM's mapping system is entirely decorator-based.
<code>reflect-metadata</code>	Runtime metadata API. Must be the first import. Enables TypeORM to read TypeScript type information (<code>"design:type"</code>) at runtime.

Interview Questions

1. JavaScript already runs in Node.js perfectly well. What concrete problem does adding TypeScript to a Node.js project solve, beyond a general claim of "type safety"?

Expected answer: TypeScript catches a specific class of bug — type mismatches, property name typos, wrong argument order, access of potentially-undefined values — at compile time rather than runtime. In a Node.js server, a runtime bug affects all concurrent requests simultaneously and may have already returned corrupted data before it is noticed. TypeScript moves that detection to the developer's machine before deployment. It also enables reliable editor autocomplete on your own code, safer large-scale refactoring, and serves as inline documentation for function contracts.

2. You install TypeScript as a `devDependency` and `reflect-metadata` as a regular dependency. Explain the reasoning behind this distinction.

Expected answer: TypeScript (`tsc` , `ts-node` , and `@types/*` packages) is only needed during

development and the build step. It produces no code that runs in production — the compiled output is plain JavaScript that Node.js executes without TypeScript being present. `reflect-metadata`, however, is imported at the top of the application entry point and its runtime code (the patched `Reflect` API) must be present when the application runs. Leaving it out of production would cause TypeORM to throw an error on startup.

3. What does `emitDecoratorMetadata: true` do in `tsconfig.json`, and what breaks in a TypeORM project if it is omitted?

Expected answer: With `emitDecoratorMetadata: true`, the TypeScript compiler emits extra `Reflect.metadata("design:type", T)` calls into the compiled JavaScript for each decorated property, recording the TypeScript type constructor (`String`, `Number`, `Boolean`, etc.). TypeORM reads this via `Reflect.getMetadata("design:type", ...)` to determine what database column type to generate. Without it, TypeORM cannot infer column types from TypeScript annotations — you would need to explicitly specify the database type in every `@Column()` call (`@Column("varchar")`, `@Column("decimal")`, etc.), and some type-dependent behaviours may fail entirely.

4. At what point does a class decorator function execute, and why does that timing matter for TypeORM?

Expected answer: A class decorator executes when the class is *defined* — when the module containing the class is first loaded. It does not execute when instances are created with `new`. This timing is essential for TypeORM: when `AppDataSource.initialize()` runs, it imports all entity modules. Loading those modules triggers all their `@Entity()` and `@Column()` decorators, which store metadata. TypeORM then reads that metadata to build the schema and set up its internal mapping tables — all before any query is made.

5. What is the difference between a TypeScript interface and a class, and when would you choose each in a TypeORM project?

Expected answer: An interface is compile-time only — it is erased from the compiled output and has no runtime existence. It cannot be instantiated, passed to `instanceof`, or used in a `require()`. A class compiles to a JavaScript constructor function that exists at runtime. In a TypeORM project: use interfaces for DTOs (describing the shape of request bodies and response objects — pure type information), and use classes for entities (they must exist at runtime so TypeORM can register decorators, read metadata, and create instances when mapping database rows).

6. Why is it important that `import "reflect-metadata"` is the first import in the application entry point?

Expected answer: `reflect-metadata` works by patching the global `Reflect` object with its metadata API (`Reflect.defineMetadata`, `Reflect.getMetadata`, etc.). If any entity module is imported before `reflect-metadata` runs, that entity's decorators will attempt to call `Reflect.defineMetadata` — which does not exist yet — and throw a runtime error. Placing the import first guarantees the API is available before any decorator can fire.

7. A colleague disables `strict: true` in `tsconfig.json` because it produces "too many errors." What are the likely consequences?

Expected answer: `strict: true` enables `strictNullChecks` and `noImplicitAny` as its two most significant sub-options. Disabling them means: (1) TypeScript no longer checks for `null` or `undefined` values — every `Cannot read properties of undefined` runtime error that TypeScript would have caught is now a live server crash waiting to happen. (2) TypeScript silently infers `any` when it cannot determine a type — effectively disabling type checking in large portions of the codebase. The "too many errors" are real bugs being surfaced; suppressing them does not fix the bugs, it hides them until production.

8. You write `import express from 'express'` in a TypeScript project. Explain what `esModuleInterop: true` does and what would happen without it.

Expected answer: Express is a CommonJS module that exports its application factory directly as `module.exports` — it has no `.default` property. With `esModuleInterop: true`, the TypeScript compiler generates a compatibility shim that checks for `module.exports.default` and falls back to `module.exports` itself. Without it, `import express from 'express'` would compile to code that looks for a `.default` property that does not exist — making `express` `undefined` at runtime and causing `express()` to throw "express is not a function". The workaround without `esModuleInterop` is the less idiomatic `import * as express from 'express'`.

Practical Assignment — Build a Decorator-Based Schema Registry

Objective: Build a TypeScript Node.js project from scratch, implementing everything in this lecture. The centrepiece is a working decorator system that mirrors what TypeORM does internally — by the end you will have built a miniature version of `@Entity()`, `@Column()`, and `@PrimaryGeneratedColumn()` from first principles.

Part A — Project Setup

1. Create a project directory called `ts-schema-demo`.
2. Initialise npm and install all packages from Section 3 with the correct dependency types (dev vs regular).
3. Create `tsconfig.json` with all eight options from Section 3.3 — do not copy-paste; type each one and state its purpose aloud to a colleague.
4. Add the three npm scripts from Section 3.4.
5. Verify: `npm run dev` should print a hello message.

Part B — TypeScript Fundamentals

1. Create `src/interfaces/dtos.ts` and define and export:
 - `CreateProductDto` with `name: string`, `price: number`, optional `description: string`.
 - `ProductResponse` extending the above with `id: number` and `createdAt: Date`.
2. Create `src/models/Product.ts` with a `Product` class using constructor shorthand. Include `public name`, `public price`, and `private sku`. Add a `describe(): string` method.
3. Import both into `src/index.ts` and use them. Log meaningful output that demonstrates both the interface shape and the class method work correctly.

Part C — The Decorator System

1. Create `src/decorators/Entity.ts` — a decorator factory that takes a table name string and stores it on the class via `Reflect.defineMetadata("tableName", ...)`.
2. Create `src/decorators/Column.ts` — a property decorator that reads `Reflect.getMetadata("design:type", ...)` and appends `"propertyName: TypeName"` to a `"columns"` metadata list on the class.

3. Create `src/decorators/PrimaryGeneratedColumn.ts` — a property decorator that stores the property name under a `"primaryKey"` metadata key on the class.
4. Create `src/entity/ProductEntity.ts` applying all three decorators, with columns: `id: number`, `name: string`, `price: number`, `isActive: boolean`.
5. In `src/index.ts`, read back all stored metadata from `ProductEntity` and print the simulated schema.

Expected Output

Output:

```
--- Part B output ---
Product: Wireless Headphones - £79.99 (SKU: WH-001)
Response: { id: 1, name: 'Wireless Headphones', price: 79.99, createdAt: [Date] }

--- Part C: Simulated schema ---
Entity:      ProductEntity
Table:       products
Primary Key: id (auto-generated)
Columns:
  id        → Number
  name      → String
  price     → Number
  isActive  → Boolean

-- This is what TypeORM reads to generate SQL. Actual TypeORM uses this same mechanism. --
```

Bonus Challenge

Add a `Nullable` property decorator that marks a column as nullable. Update the schema print to show `(nullable)` next to any column that has it. Then open the 11-01 notes and compare your three decorators to TypeORM's actual `@Entity()`, `@PrimaryGeneratedColumn()`, and `@Column({ nullable: true })` implementations. The structural similarities should now be clear.